

koroteevmv / ML_course

<> Code

Issues 2

Pull requests

Agents

Actions

Projects

Security and quality

Insights

Files

main

Go to file

ML0.1 intro

ML0.2 simple analysys

ML1.1 linear regression

ML1.2 multivariate

ML1.3 real regression

ML2.1 logistic regression

ML2.2 real classification

ML3.1 polynomial features

ML3.2 svm

ML3.3 mlp

ML3.4 trees

ML3.5 knn

ML3.6 bayes

ML4.1 metrics

ML4.2 threshold

ML4.3 cross-validation

ML4.4 diagnostics

ML4.5 hyperparameters

ML 5.1 data interanction

ML_course / ML1.2 multivariate /

Add file

...

koroteev-test

Картинки распределены по папкам

7dd1f73 · 2 years ago

History

Name	Last commit message	Last commit date
..		
data	new structure created	3 years ago
img	Картинки распределены по папкам	2 years ago
README.md	Картинки распределены по папкам	2 years ago

README.md

Множественная линейная регрессия на чистых данных

Цель работы

Познакомиться с основными приемами работы с множественными регрессионными моделями с использованием библиотеки sklearn.

Содержание работы

- Ваша задача - построить предиктивную модель предсказания целевой переменной в зависимости от значения нескольких факторов (признаков).
- Загрузите датасет из приложения. Вам понадобится два файла - отдельно с признаками (помечен x) и значениями целевой переменной (помечен y). Файлы не содержат заголовков столбцов и строк. Выведите первые строки датасета.
- Сделайте количественное описание датасета: количество точек данных, признаков, статистику по каждому признаку.
- Для целевой и каждого из признаков постройте парную регрессионную модель. Визуализируйте линию регрессии и исходные данные на графике. При помощи коэффициентов оцените силу связи.
- Постройте модель множественной линейной регрессии целевой переменной от всех показателей. Оцените качество регрессии, сделайте вывод.

Методические указания

Отчет по лабораторной работе сдается в виде файла интерактивного ноутбука Jupyter notebook и должен включать весь программный код, реализующий задания лабораторной работы, подробные пояснения, обоснования и выводы там, где это требуется в виде текстовых ячеек.

Для начала работы как всегда выполним стандартный импорт необходимых библиотек:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Теперь мы можем приступить к чтению данных. Для этой работы представлено сразу 50 однотипных датасетов. Мы для примера возьмем самый первый (с названием "0"). Читать данные можно многими способами, но мы воспользуемся самым простым чтением по URL:

```
x = pd.read_csv("https://github.com/koroteevmv/ML_course/raw/main/ML1.2%20multivariate/data/0_x.csv")
```

Теперь нужно вывести датасет (хотя бы первые несколько строк) на экран, чтобы убедиться, что все прочиталось нормально:

```
x.head()
```

Если выполнить этот код то при внимательном рассмотрении можно заметить одну ошибку. Первая строка данных была помещена в заголовки строк. Чаще всего в файле с данными первая строка как раз и содержит заголовки, поэтому такое поведение чаще бывает полезным. Но в нашем случае датасет не содержит заголовков. Поэтому при чтении надо это учесть:

```
x = pd.read_csv("https://github.com/koroteevmv/ML_course/raw/main/ML1.2%20multivariate/data/0_x.csv",
                header=None)
y = pd.read_csv("https://github.com/koroteevmv/ML_course/raw/main/ML1.2%20multivariate/data/0_y.csv",
                header=None)
```

Теперь данные прочтены верно. Но лучше еще раз в этом убедиться, выведя часть датасета на экран.

При работе с данными важно понимать размер данных и то, как они располагаются в структуре, то есть в переменных. Выведем количественные параметры датаета:

```
>>> x.shape, y.shape
((387, 5), (387, 1))
```

Здесь мы должны заметить сразу несколько вещей. Во-первых, обе переменные представляют собой двумерные массивы. Точнее они представлены в виде DataFrame. В матрице атрибутов (X) 387 строк и 5 столбцов. Это значит, что в датасете описаны 387 объектов и у каждого есть 5 признаков. Или, можно сказать, что в X 5 векторов признаков по 387 значений. Важно то, что по строкам у нас объекты, а по столбцам - атрибуты (характеристики).

В целевой переменной (Y) у нас один столбец и 387 строчек. Очень важно, чтобы количество строк в обоих переменных совпадало. Фактически, Y представляет собой вектор-столбец. То есть один единственный атрибут описан для 387 объектов.

Такое представление данных подходит для начала машинного обучения. Все, что нужно совпадает. Теперь мы можем приступить к созданию и обучению модели. Для этого сначала импортируем нужный объект из библиотеки scikit-learn:

```
from sklearn.linear_model import LinearRegression
```

В данном случае нам нужна модель линейной регрессии, которая является частью пакета "linear_model" и называется LinearRegression. Про другие модели и пакеты данной библиотеки можно узнать из документации.

Создадим объект модели:

```
binary = LinearRegression()
```

Для начала мы построим модель парной регрессии от одного из признаков в датасете. Для этого из матрицы атрибутов нужно вытащить один столбец. Можно сделать это так: `x[1]`. Напоминаем, что в DataFrame по умолчанию индексация происходит по столбцам. Но тут возникает проблема. Дело в том, что все библиотечные средства предполагают, что X - это двумерный массив. А взятие одного столбца возвращает не DataFrame, а Series - одномерный массив. Превратить его в двумерный можно многими разными способами, но в Pandas можно взять несколько столбцов из датафрейма. Такая операция вернет всегда именно датафрейм. А нам нужен датафрейм из одного столбца. Сделать это можно так: `x[[1]]`. Итак, возьмем один столбец и обучим парную регрессию его с целевой переменной:

```
binary.fit(x[[1]], y)
```

Данный метод подстроил коэффициенты линейной модели таким образом, чтобы функция модели была как можно ближе к точкам. Сами коэффициенты тоже можно посмотреть. Получаем коэффициенты b_n :

```
binary.coef_
```

Метод predict() рассчитывает теоретические значения признака:

```
binary.predict(x[[1]])
```

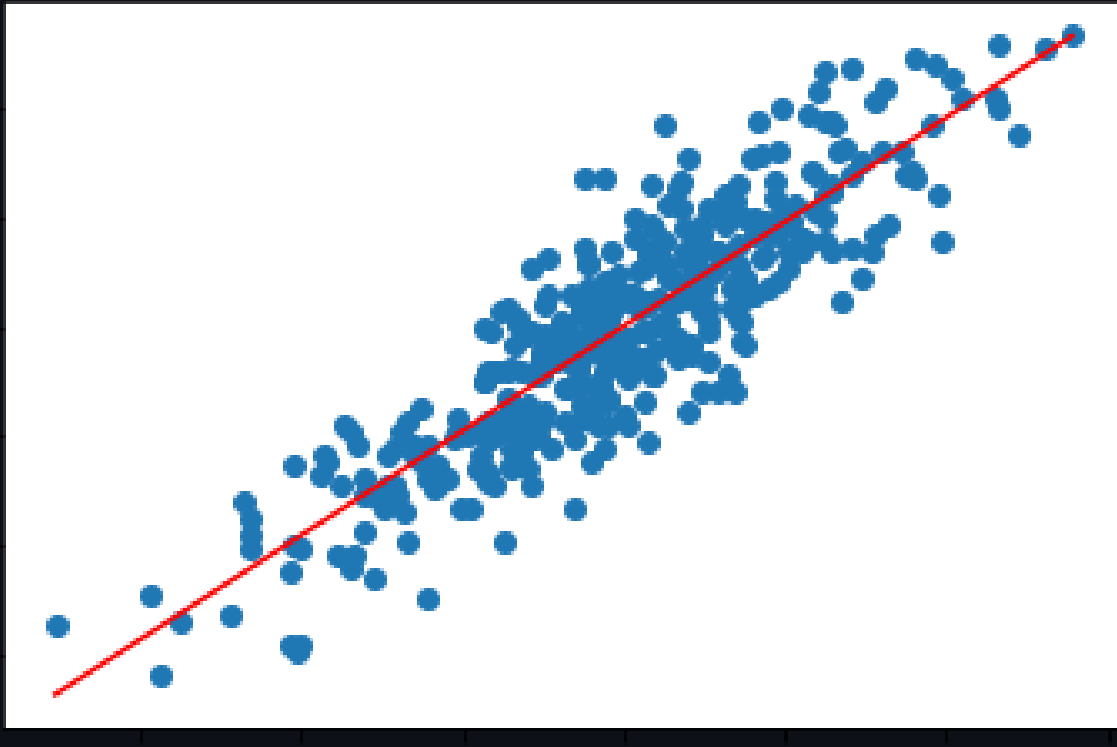
Конечно, вручную сравнивать их с реальными (эмпирическими) не нужно. Гораздо проще построить их на графике. Построить точки очень просто:

```
plt.scatter(x[1], y)
```

Для построения линии модели просто использовать plot вот так: `plt.plot(x[1], y)` будет уже неправильно. Лучше построить равномерное распределение точек от минимального до максимального по X и уже по нему рассчитать модельные значения:

```
xx = np.linspace(x[1].min(), x[1].max(), 100).reshape((-1, 1))
plt.plot(xx, binary.predict(xx), c='r')
```

Вот что получится в итоге:



Для оценки качества построенной регрессии можно использовать целый ряд показателей. Одним из самых простых и универсальных является коэффициент детерминации (R2-score). Как использовать его читайте в [официальной документации sklearn](#). Методом score можно узнать значение метрики эффективности полученной модели:

```
reg.score(X, Y)
```

В нашем случае оценка будет примерно 0.77, что свидетельствует о средней точности модели.

Для построения множественной регрессии выполним те же шаги, но будем использовать полный датасет:

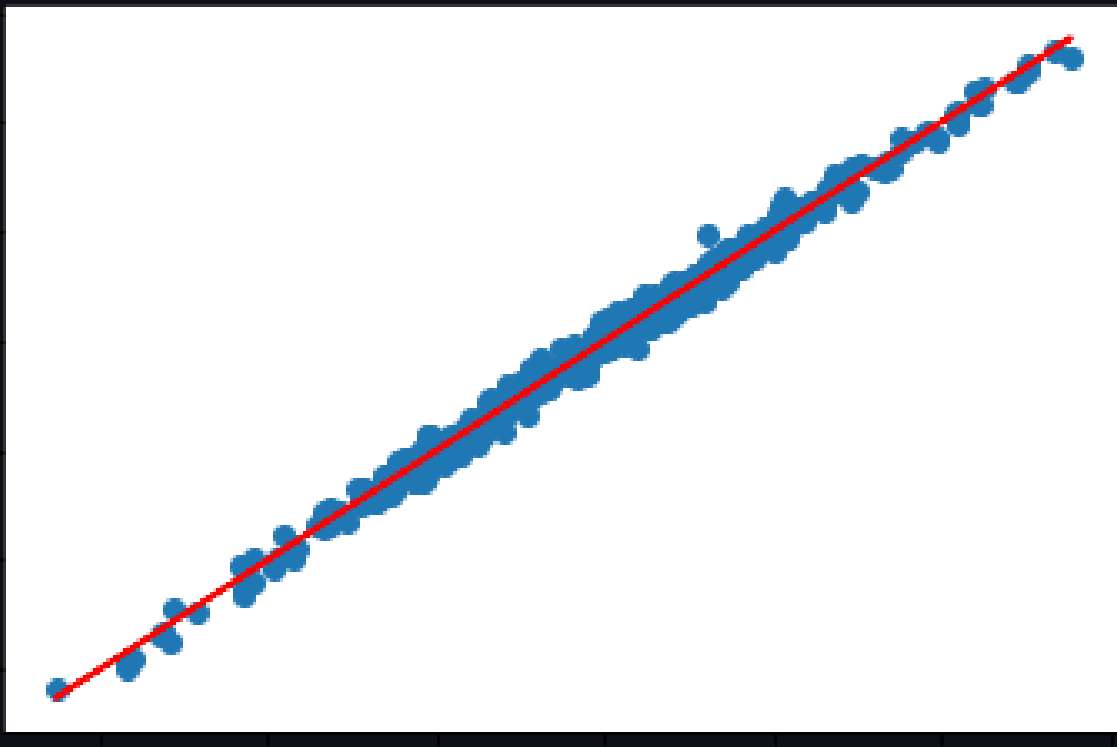
```
multiple = LinearRegression()
multiple.fit(x, y)
multiple.score(x, y)
```

Теперь точность возросла до 0.99, что говорит о почти идеальной модели. Вообще чем больше информации мы подаем на вход моделям машинного обучения, тем на более сильный результат мы можем рассчитывать.

В модели множественной регрессии уже не удастся построить такой красивый и однозначный график. Для того, чтобы хоть как-то увидеть, как модель соотносится с точками, можно построить график зависимости теоретических значений от эмпирических, то есть как предсказанные значения отличаются от истинных:

```
yy = multiple.predict(x)
plt.scatter(yy, y)
plt.plot(yy, yy, c='r')
```

Дополнительно на этом графике мы добавили линию $y=x$. Это всегда будет прямая, даже если мы будем использовать нелинейные модели. Обратите внимание, что на данном графике по обоим осям Y - это целевая переменная:



Такой график наряду с диагностическими кривыми можно использовать для диагностики многомерных задач регрессии.

Задания для самостоятельного выполнения

- Самостоятельно постройте парные регрессии от каждого атрибута с целевой переменной.
- (*) Модифицируйте код из предыдущей работы так, чтобы модель регрессии умела работать с входными данными, состоящими из произвольного количества столбцов.
- Для целевой переменной и каждого из признаков постройте полиномиальную регрессию второго, третьего и десятого порядков. Сделайте вывод о значимости регрессии.
- Постройте сводную таблицу показателей точности всех построенных моделей. Сделайте вывод, какие модели переобученные, какие - недообученные.
- Используйте другие метрики качества регрессионной модели в дополнение к коэффициенту детерминации.
- (*) Замерьте время обучения полиномиальных моделей с разной степенью полинома. Сделайте вывод.

Контрольные вопросы

- Как должны выглядеть данные для модели машинного обучения?
- В чем должен смысл коэффициентов линейной регрессии?
- Чем множественная регрессия отличается от парной?
- Какую функцию ошибки использует модель линейной регрессии?
- Какие метрики эффективности можно применять для оценки регрессионных моделей?
- Какие еще регрессоры есть в библиотеке sklearn?
- Почему для визуализации модели нельзя использовать предсказанные значения по датасету?

Дополнительные задания

- Для целевой переменной и каждого из признаков постройте полиномиальную регрессию второго, третьего и десятого порядков. Сделайте вывод о значимости регрессии.
- (*) Модифицируйте класс модели из предыдущей лабораторной работы таким образом, чтобы он работал в режиме множественной регрессии. Для этого на вход ему в качестве X должен подаваться датафреймиз произвольногоколичества строк.